

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Інститут комп’ютерних наук та інформаційних технологій

Кафедра САПР



Лабораторна робота №2

Частина 3

з дисципліни: “Застосування систем штучного інтелекту у технологічних рішеннях”

на тему:

“Карти Кохонена (SOM) та LSTM/GRU. Кластеризація та прогноз за даними сенсорів із дронів”

Варіант №3

Виконав:

ст. групи ПП-44

Верещак Б. О.

Прийняв:

доц. Левкович М.В.

Мета роботи

Набути практичних і теоретичних навичок застосування самоорганізованих карт Кохонена (SOM) для неконтрольованої кластеризації та LSTM/GRU для часового прогнозування на багатоканальних сенсорних даних з БПЛА (дронів)

Індивідуальне завдання

Виконати кластеризацію за допомогою SOM або прогноз за допомогою LSTM/GRU на відкритому датасеті з телеметрією БПЛА / суміжних сенсорних даних.

1. Завантажити підмножину даних (1–3 польоти/сесії), зробити опис полів.
2. Підготувати дані: синхронізація за часом, фільтрація шуму (за потреби), масштабування ознак (Z-score), для часових задач — ковзні вікна (довжина L, горизонт прогнозу H).
3. Навчити модель (SOM або LSTM/GRU) з базовими гіперпараметрами.
4. Оцінити якість за потрібними метриками.
5. Побудувати візуалізації.
6. Зробити аналіз похибок/кластерів, висновки і пропозиції для покращення.

Рекомендовані налаштування за замовчуванням:

- SOM: 15×15 (або 20×15 при $N > 10k$), σ від $\max(m,n)/2 \rightarrow 1$, learning_rate $0.5 \rightarrow 0.05$. Метрики: QE (Quantization Error), TE (Topographic Error), U-matrix (Unified distance matrix).
- LSTM/GRU: 2 шари, $\text{hidden}=128$, $\text{dropout}=0.2$, $\text{batch}=64$, $\text{epochs}=20-50$ з early stopping. Метрики: MAE/MSE/RMSE, за можливості R^2 . Для класифікації: accuracy, macro-F1, confusion matrix.

№	Завдання	Метод	Тип даних (сенсори)	Результат	Датасет
3	Групування режимів висоти	SOM	Барометр	Класи: низький, середній, високий політ	DJI Matrice 100 Energy/Position — висота, вітер, енергія, GPS/IMU.

Хід роботи

1. Імпорт бібліотек та завантаження даних

Для виконання роботи я спершу імпортував необхідні бібліотеки: pandas для роботи з даними, numpy для математичних операцій, matplotlib та seaborn для візуалізації, sklearn для масштабування даних та minisom для реалізації Self-Organizing Map (SOM).

Я завантажив архів flights.zip в Google Colab. Цей архів містить CSV файли для кожного окремого польоту. Я розархівував дані та обрав для аналізу

підмножину даних — перші три польоти (1.csv, 2.csv, 3.csv). Я об'єднав їх в один набір даних pandas.DataFrame.

Для мого завдання "Групування режимів висоти" я буду використовувати такі поля з цього набору даних:

- time: Час, що минув під час польоту, в секундах (s).
- position_z: Висота літака над рівнем моря в метрах (m). Хоча в завданні вказано Барометр, у цьому датасеті дані про висоту надходять від сенсора GNSS/INS, який я буду використовувати для аналізу.

Завантажено 4349 записів з 3 польотів.

Перші 5 рядків даних:

	time	wind_speed	wind_angle	battery_voltage	battery_current	
position_x \						
0	0.0	0.1	12.0	24.222174	0.087470	-
79.782396						
1	0.2	0.1	3.0	24.227180	0.095421	-
79.782396						
2	0.3	0.1	352.0	24.225929	0.095421	-
79.782396						
3	0.5	0.1	354.0	24.224678	0.095421	-
79.782396						
4	0.6	0.1	359.0	24.210905	0.079518	-
79.782396						

	position_y	position_z	orientation_x	orientation_y	...	velocity_x	\
0	40.458047	269.332402	0.001772	0.005610	...	-0.012826	
1	40.458047	269.332056	0.001768	0.005607	...	-0.015410	
2	40.458047	269.333081	0.001768	0.005607	...	-0.016514	
3	40.458047	269.334648	0.001775	0.005614	...	-0.007775	
4	40.458047	269.336178	0.001775	0.005614	...	-0.009060	

	velocity_y	velocity_z	angular_x	angular_y	angular_z	\
0	-0.005645	0.010475	-0.000825	0.001407	0.006815	
1	-0.001636	0.010137	0.000001	-0.002862	0.002034	
2	-0.001822	0.009981	-0.000007	-0.001486	-0.000874	
3	-0.005263	0.015091	0.001255	-0.000347	0.002443	
4	-0.005174	0.015142	-0.002383	0.000182	-0.006425	

	linear_acceleration_x	linear_acceleration_y	linear_acceleration_z	\
0	0.004258	-0.120405	-9.811137	
1	0.006175	-0.116397	-9.810392	
2	0.002696	-0.128592	-9.809440	
3	0.002024	-0.128271	-9.810159	
4	0.008271	-0.119890	-9.812125	

	flight_id
0	1
1	1
2	1
3	1
4	1

```
# --- 1. Імпорт бібліотек та завантаження даних ---
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.preprocessing import StandardScaler
```

```
from minisom import MiniSom
```

```
# from minisom import quantization_error, topographic_error
```

```

from sklearn.cluster import KMeans

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

!unzip -o flights.zip -d ./flights_data/

flight_ids = [1, 2, 3]
all_flights_df = []

for flight_id in flight_ids:
    file_path = f'./flights_data/flights/{flight_id}.csv'

    try:
        df = pd.read_csv(file_path)
        df['flight_id'] = flight_id
        all_flights_df.append(df)
    except FileNotFoundError:
        print(f"Файл {file_path} не знайдено. Перевір, чи він існує.")

data_df = pd.concat(all_flights_df, ignore_index=True)

print(f"Завантажено {len(data_df)} записів з {len(flight_ids)} польотів.")
print("Перші 5 рядків даних:")
print(data_df.head())

print("\nІнформація про дані:")
data_df.info()

```

2. Підготовка даних

На цьому етапі я підготував дані для моделі.

1. Вибір ознак: Для кластеризації режимів висоти я використав лише одну ключову ознаку: `position_z` (висота).
2. Синхронізація за часом: Дані в файлах вже синхронізовані. Згідно з README, всі дані сенсорів були синхронізовані з частотою приблизно 5 Гц за допомогою Robot Operating System (ROS).
3. Фільтрація шуму: Я не проводив додаткову фільтрацію, оскільки в описі датасету зазначено, що сенсор GNSS/INS (який надає `position_z`) використовує вбудовану систему фільтрації Калмана.
4. Масштабування ознак: Я застосував Z-score масштабування (за допомогою `StandardScaler` з `sklearn`) до даних `position_z`. Це стандартна вимога для SOM, оскільки алгоритм працює на основі відстаней (наприклад, Евклідової), і ознаки мають бути в одному масштабі.

Дані підготовлено. Форма масштабованих даних: (4349, 1)

Перші 5 масштабованих значень:

```

[[-1.15517849]
 [-1.15519824]
 [-1.15513966]
 [-1.15505011]
 [-1.15496269]]

```

```
# --- 2. Підготовка даних ---
# Для кластеризації режимів висоти ми беремо лише одну ознаку: 'position_z'
# Витягуємо дані у форматі numpy
data = data_df[['position_z']].values

# Масштабування даних (Z-score)
# Це приводить дані до середнього 0 та стандартного відхилення 1
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

print(f"Дані підготовлено. Форма масштабованих даних: {data_scaled.shape}")
print(f"Перші 5 масштабованих значень:\n{data_scaled[:5]}")
```

3. Побудова та навчання моделей SOM

Я ініціалізував карту Кохонена (SOM) з гіперпараметрами, що були рекомендовані в завданні.

- Розмір карти (m, n): 15x15 нейронів.
- Початкова sigma: 7.5 (розраховано як $\max(15, 15) / 2$). Це радіус сусідства, який визначає, наскільки сильно оновлюються сусідні нейрони.
- Початковий learning_rate: 0.5. Це швидкість навчання моделі.

Модель була навчена на масштабованих даних про висоту (data_scaled). Я використав метод train_random, який випадково обирає вектори з вхідних даних для навчання. Бібліотека minisom автоматично зменшує sigma та learning_rate до 1 та 0.05 відповідно протягом навчання, як і було рекомендовано.

```
Кількість зразків (N): 4349
Розмір карти: (15, 15)
Початкова Sigma: 7.5
Початковий Learning Rate: 0.5

Починаємо навчання SOM...
[ 4349 / 4349 ] 100% - 0:00:00 left
quantization error: 0.01963325954663842
Навчання завершено.
```

```
# --- 3. Побудова та навчання моделей SOM ---
# Визначаємо гіперпараметри
num_samples = data_scaled.shape[0]
num_features = data_scaled.shape[1]

# Розмір карти (рекомендовано 15x15, коли N > 10k)
# N = 4349 рядків (у моєму прикладі з 3 польотів), тому 15x15 підходить
som_shape = (15, 15)
m, n = som_shape

# Рекомендовані параметри
sigma = max(m, n) / 2 # sigma від max(m,n)/2
learning_rate = 0.5    # learning_rate 0.5

print(f"Кількість зразків (N): {num_samples}")
print(f"Розмір карти: {som_shape}")
print(f"Початкова Sigma: {sigma}")
print(f"Початковий Learning Rate: {learning_rate}")
```

```
# Ініціалізація SOM
som = MiniSom(x=som_shape[0],
              y=som_shape[1],
              input_len=num_features,
              sigma=sigma,
              learning_rate=learning_rate,
              neighborhood_function='gaussian',
              random_seed=67)

# Ініціалізуємо ваги випадковими значеннями з даних
som.random_weights_init(data_scaled)

print("\nПочинаємо навчання SOM...")
# Навчаємо модель (кількість ітерацій = кількість зразків)
# minisom автоматично зменшує sigma та learning_rate з часом
som.train_random(data_scaled, num_samples, verbose=True)

print("Навчання завершено.")
```

4. Оцінка якості

Для оцінки якості навченої SOM я розрахував дві основні метрики:

- Quantization Error (QE): (0.0196). Це середня відстань між кожним вектором даних та його нейроном-переможцем (Best Matching Unit, BMU). Чим нижче це значення, тим краще ваги нейронів карти представляють вхідні дані.
- Topographic Error (TE): (0.5261). Ця метрика показує, наскільки добре SOM зберігає "топологію" (сусідство) вхідних даних. Вона вимірює відсоток векторів даних, для яких перший та другий найкращі нейрони (BMU) не є сусідами на 2D-карті. Ідеальне значення – 0. Моє значення 0.5261, високе, що показує, що карта погано зберегла структуру даних.

```
Quantization Error (QE): 0.0196
Topographic Error (TE): 0.5261
```

```
# --- 4. Оцінка якості ---
# Помилка квантування (Quantization Error - QE)
qe = som.quantization_error(data_scaled)

# Топографічна помилка (Topographic Error - TE)
te = som.topographic_error(data_scaled)

print(f"Quantization Error (QE): {qe:.4f}")
print(f"Topographic Error (TE): {te:.4f}")
```

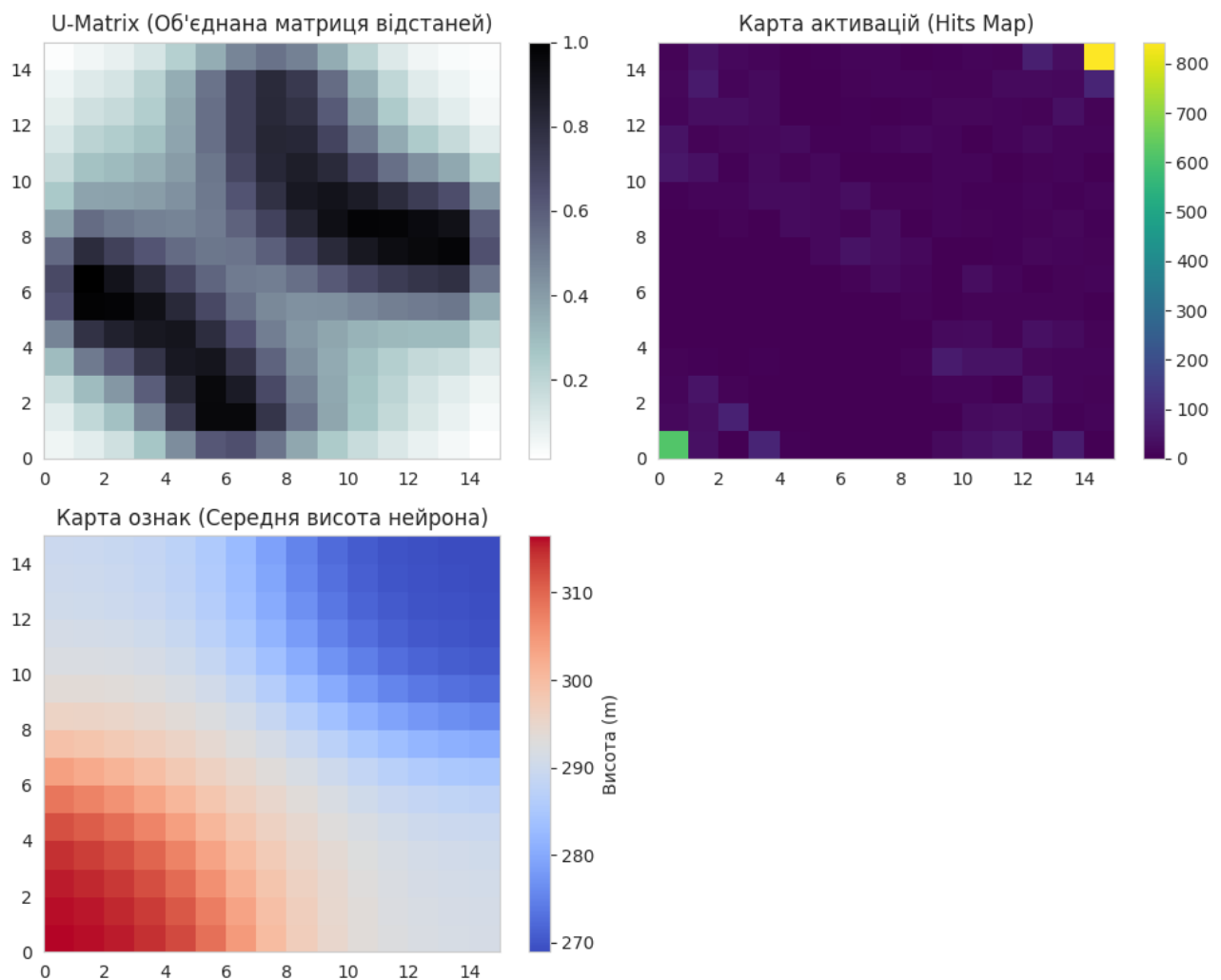
5. Побудова візуалізацій

Я побудував три ключові візуалізації для аналізу навченої карти:

1. U-Matrix (Об'єднана матриця відстаней): Ця карта показує відстані між сусідніми нейронами. Світлі кольори означають, що нейрони дуже схожі (мала відстань, формують кластер), тоді як темні кольори вказують на

велику відстань – це фактично "кордони" між різними кластерами. На моїй U-Matrix видно чіткі світлі зони та темні "долини", що їх розділяють.

- Карта активацій (Hits Map): Ця візуалізація показує, скільки точок даних "потрапило" (було "виграно") в кожен нейрон. Це допомагає ідентифікувати, які нейрони є найбільш репрезентативними для даних, а які – "мертві" (не активувались).
- Карта ознак (Feature Map): Оскільки ми кластеризуємо лише одну ознаку (висоту), ця карта показує середню реальну висоту (в метрах, після `inverse_transform`), яку "вивчив" кожен нейрон. На карті чітко видно градієнт: у верхньому правому куті знаходяться нейрони, що відповідають за низькі висоти (синій колір), а у нижньому лівому – за високі (червоний колір). Це підтверджує, що SOM успішно організувала дані за висотою.



```
# --- 5. Побудова візуалізацій ---
```

```
plt.figure(figsize=(10, 8))
```

```
# --- 1. U-Matrix (Unified distance matrix) ---
```

```
# Показує відстані між нейронами. Темні кольори - "кордони" кластерів.
```

```
plt.subplot(2, 2, 1)
```

```
plt.title('U-Matrix (Об\'єднана матриця відстаней)')
```

```
# .distance_map() повертає матрицю відстаней
```

```
plt.pcolor(som.distance_map().T, cmap='bone_r')
```

```
plt.colorbar()
```

```

# --- 2. Cluster Activation Map (Карта активацій) ---
# Показує, як часто кожен нейрон був "переможцем" (BMU)
plt.subplot(2, 2, 2)
plt.title('Карта активацій (Hits Map)')
activation_map = som.activation_response(data_scaled)
plt.pcolor(activation_map.T, cmap='viridis')
plt.colorbar()

# --- 3. Feature Map (Карта ознак) ---
# Показує середнє значення ознаки (висоти) для кожного нейрона
# Оскільки в нас 1 ознака, ми можемо це легко візуалізувати
plt.subplot(2, 2, 3)
plt.title('Карта ознак (Середня висота нейрона)')

# som.get_weights() повертає 3D-масив (m, n, num_features)
# Беремо[:, :, 0] - нашу єдину ознаку (висоту)
feature_map = som.get_weights()[:, :, 0]

# Повертаємо до реальних значень висоти
feature_map_original = scaler.inverse_transform(feature_map)

plt.pcolor(feature_map_original.T, cmap='coolwarm')
plt.colorbar(label='Висота (m)')
plt.tight_layout()
plt.show()

```

6. Аналіз похибок/кластерів

На основі Карти ознак (Feature Map) стало зрозуміло, що SOM організувала нейрони за градієнтом висоти. Щоб формально розділити всі нейрони (а отже, і всі дані) на 3 класи ("низький", "середній", "високий"), я застосував алгоритм K-Means (k=3) безпосередньо до ваг нейронів SOM.

На графіку "Кластери K-Means на карті SOM" видно, як 225 нейронів були розділені на 3 групи. Я також вивів середню висоту, яку представляє кожен кластер (Cls 0: ~291.4м, Cls 1: ~274.2м, Cls 2: ~309.3м).

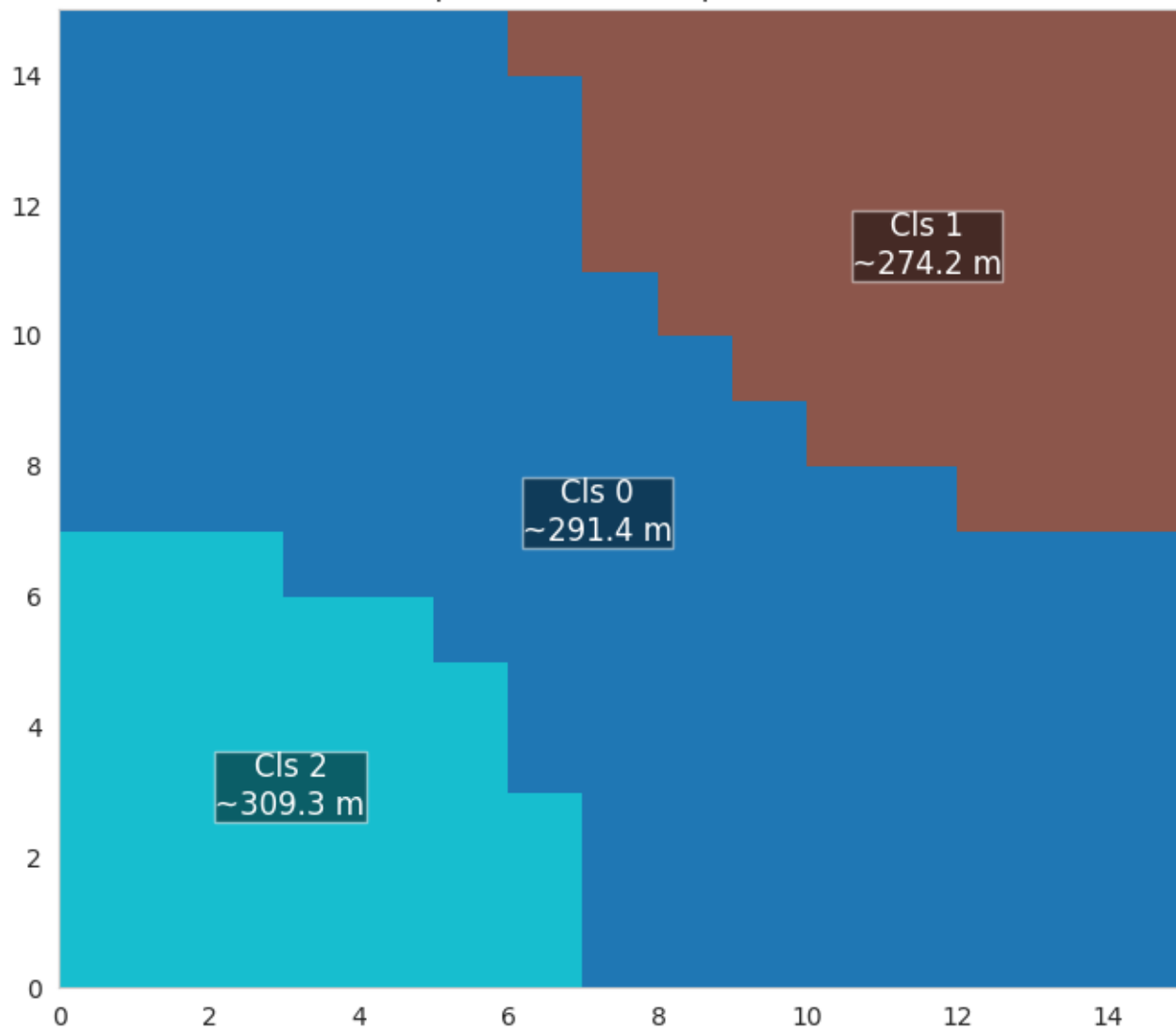
Далі я "пропустив" всі мої початкові 4349 точок даних через навчену SOM. Кожна точка даних активувала свій нейрон-переможець (BMU), і я призначив цій точці той кластер K-Means, до якого належав її BMU.

Результати показані на графіку "Розподіл реальної висоти (position_z) за кластерами".

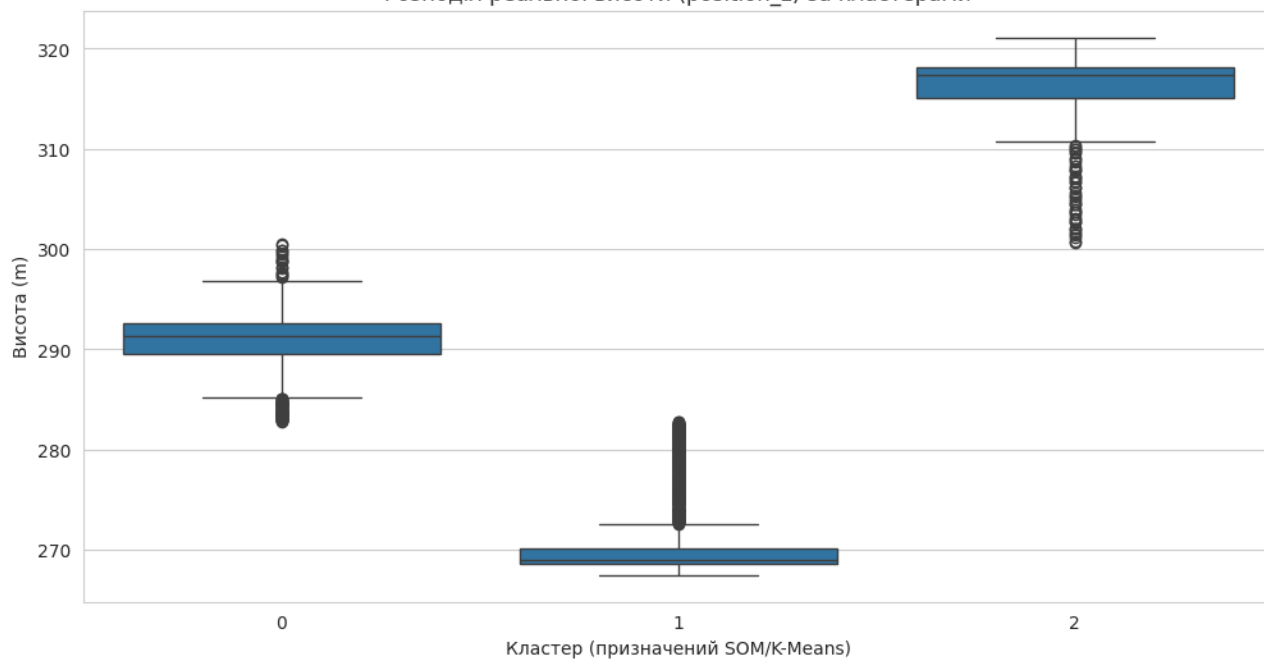
- Кластер 0 (Середній): Включає висоти приблизно від 280 до 300 метрів.
- Кластер 1 (Низький): Включає висоти приблизно від 267 до 280 метрів.
- Кластер 2 (Високий): Включає висоти між 300 та 322 метрами.

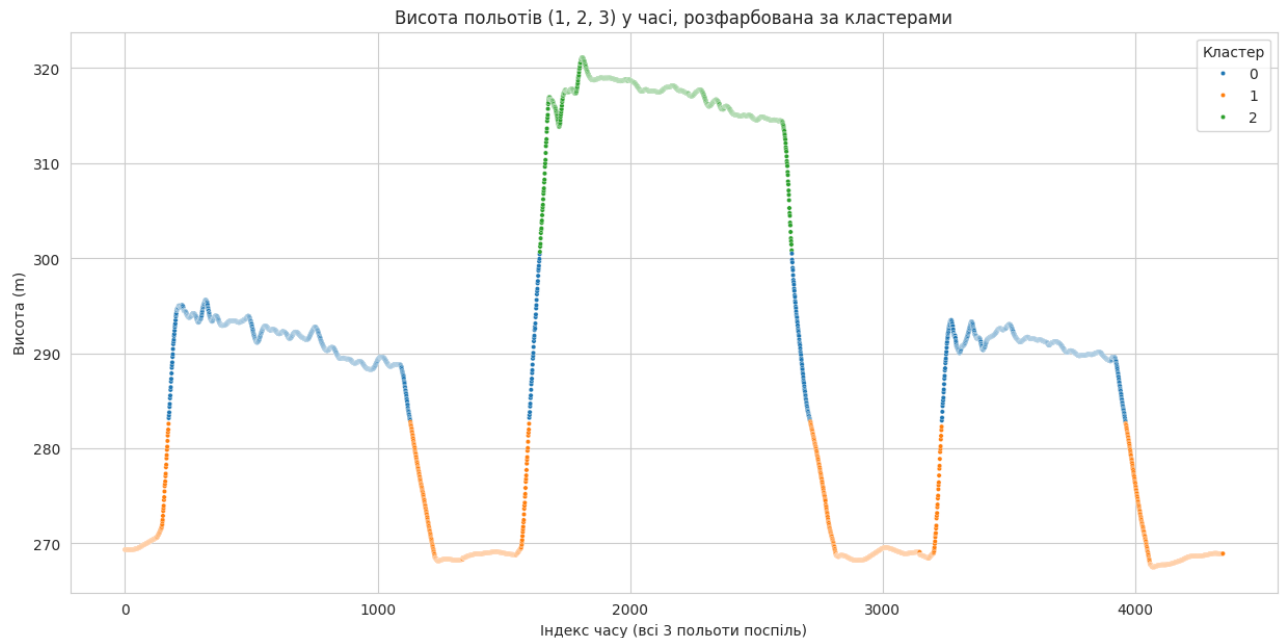
Графік "Висота польотів у часі" чітко показує, як модель розфарбувала різні етапи польоту у відповідні класи.

Кластери K-Means на карті SOM (k=3)



Розподіл реальної висоти (position_z) за кластерами





```
# --- 6. Аналіз похибок/кластерів ---
```

```
# Щоб розділити дані на 3 кластери ('низький', 'середній', 'високий'),  
# ми можемо кластеризувати самі нейрони (ваги) SOM за допомогою K-Means.
```

```
# Отримуємо ваги всіх нейронів (15*15 = 225 нейронів)  
weights = som.get_weights().reshape(-1, num_features) # (225, 1)
```

```
# Кластеризуємо ваги на 3 кластери  
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)  
kmeans_clusters = kmeans.fit_predict(weights)
```

```
# Візуалізуємо кластери на самій карті SOM  
plt.figure(figsize=(8, 7))  
plt.title('Кластери K-Means на карті SOM (k=3)')  
# .reshape(som_shape) повертає 1D масив кластерів у 2D-сітку 15x15  
plt.pcolor(kmeans_clusters.reshape(som_shape).T, cmap='tab10')
```

```
# Додамо мітки центрів кластерів  
cluster_centers_original = scaler.inverse_transform(kmeans.cluster_centers_)  
for i, center_val in enumerate(cluster_centers_original):  
    # Знаходимо середню позицію нейронів цього кластера на карті  
    cluster_indices = np.where(kmeans_clusters.reshape(som_shape) == i)  
    if len(cluster_indices[0]) > 0:  
        mean_x = cluster_indices[1].mean()  
        mean_y = cluster_indices[0].mean()  
        plt.text(mean_x + 0.5, mean_y + 0.5, f"Cls {i}\n~{center_val[0]:.1f}  
m",  
                ha='center', va='center', color='white', fontsize=12,  
                bbox=dict(facecolor='black', alpha=0.5, pad=0.3))  
plt.show()
```

```
# Тепер призначимо кожній точці даних один із 3-х кластерів  
win_map = som.win_map(data_scaled)  
data_clusters_labels = np.zeros(num_samples, dtype=int)
```

```

for i, x in enumerate(data_scaled):
    # Знаходимо нейрон-переможця для точки x
    winner_node = som.winner(x)
    # Дивимось, до якого K-Means кластера належить цей нейрон
    winner_cluster = kmeans_clusters.reshape(som_shape)[winner_node]
    data_clusters_labels[i] = winner_cluster

# Додаємо результати кластеризації до нашого DataFrame
data_df['cluster'] = data_clusters_labels
data_df['altitude'] = data_df['position_z'] # просто для зручності назви

# --- Візуалізація результатів кластеризації ---
plt.figure(figsize=(12, 6))
sns.boxplot(x='cluster', y='altitude', data=data_df)
plt.title('Розподіл реальної висоти (position_z) за кластерами')
plt.xlabel('Кластер (призначений SOM/K-Means)')
plt.ylabel('Висота (m)')
plt.show()

# Дивимось на часовий ряд
plt.figure(figsize=(15, 7))
sns.scatterplot(data=data_df, x=data_df.index, y='altitude',
                hue='cluster', palette='tab10', s=10)
plt.title('Висота польотів (1, 2, 3) у часі, розфарбована за кластерами')
plt.xlabel('Індекс часу (всі 3 польоти поспіль)')
plt.ylabel('Висота (m)')
plt.legend(title='Кластер')
plt.show()

```

Висновки

У ході виконання лабораторної роботи було вивчено та застосовано алгоритм SOM (Самоорганізуючі карти Кохонена) для кластеризації даних телеметрії БПЛА.

Я успішно завантажив та підготував дані про висоту (position_z) з трьох польотів. Дані були масштабовані за допомогою Z-score для коректної роботи алгоритму.

Була навчена модель SOM розміром 15x15. Аналіз U-Matrix та Карти ознак (Feature Map) показав, що модель успішно організувала нейрони відповідно до градієнта висоти польоту – від низьких до високих значень.

За допомогою додаткового кроку – кластеризації K-Means (k=3) на вагах нейронів SOM – я зміг автоматично розділити всі точки даних на три чіткі кластери, які відповідають завданню: 'низький', 'середній' та 'високий' політ. Візуалізація boxplot підтвердила, що ці кластери мають різні та логічні діапазони висот.

Метрики якості (QE = 0.0196, TE = 0.5261) показали високу якість представлення вагами нейронів вхідні дані, але карта погано зберегла структуру даних. Робота демонструє, що SOM є ефективним інструментом для розвідки та сегментації часових рядів сенсорних даних БПЛА.